Les Structures Répétitives en Algorithmique et Langage C : Boucles While, Do-While et For

Thème principal : Structures itératives et automatisation en programmation

Synthèse exécutive : Ce cours aborde les structures répétitives (boucles) en algorithmique et en langage C. L'enjeu fondamental est d'automatiser l'exécution d'un bloc d'instructions à répétition sans recourir à la duplication de code, en respectant scrupuleusement le principe DRY (Don't Repeat Yourself). Sont étudiées la boucle while (pré-condition, de 0 à N itérations, idéale pour les événements dynamiques), la boucle do-while (post-condition, 1 à N itérations garanties, indispensable pour les contrôles de saisie) et la boucle for (itérations déterminées, centralisant initialisation, condition et incrémentation). Une attention particulière est portée sur la prévention des boucles infinies, la gestion des variables de contrôle et les pièges syntaxiques fréquents en langage C.

Utilité des structures itératives et principe DRY

En programmation algorithmique et en langage C, la répétition d'instructions ne doit jamais être gérée par du copier-coller. Si l'on doit traiter 10 000 éléments d'un tableau ou redemander une saisie utilisateur invalide, dupliquer manuellement les lignes de code détruit la maintenabilité, alourdit inutilement l'exécutable et constitue une source massive de bogues. Les structures itératives (ou boucles) permettent de répéter l'exécution d'un bloc d'instructions tant qu'une condition logique (expression booléenne) reste vraie. L'enjeu central de la conception d'une boucle repose sur la maîtrise de la condition d'arrêt et l'actualisation effective de la variable de contrôle à chaque itération. Si le corps de la boucle n'altère pas les variables testées dans la condition, celle-ci reste indéfiniment vraie, provoquant une boucle infinie qui gèle l'exécution.

- Principe DRY (Don't Repeat Yourself) : interdiction absolue de dupliquer du code par copier-coller.
- Condition d'arrêt : expression booléenne évaluée à chaque passage pour déterminer la poursuite ou l'interruption de la boucle.
- Variable de contrôle / Compteur : variable modifiée au sein du corps de la boucle pour faire évoluer la condition d'arrêt vers Faux.
- Boucle infinie : dysfonctionnement critique où la condition reste indéfiniment Vraie, bloquant l'exécution du programme.

Cas concret enseignant : *Métaphore du tapis roulant d'usine d'emballage : tant qu'un colis est détecté par le capteur (condition vraie), le bras robotisé l'attrape et l'emballe. Si le capteur ne détecte plus de colis ou si le quota fixé est atteint, le robot s'arrête. Sans condition de fin fonctionnelle, le robot continuerait à tourner à vide et finirait par surchauffer.*

La boucle TANT QUE (while) : Structure à pré-condition

La structure while est la boucle de référence lorsque le nombre exact d'itérations n'est pas connu à l'avance et dépend d'un événement dynamique externe (ex: une saisie utilisateur, la lecture d'un fichier jusqu'à la fin, ou la réception d'un signal réseau). Puisque le test est réalisé avant toute exécution, si la condition s'avère fausse lors du premier contrôle, le corps de la boucle est totalement ignoré (0 itération). Il faut être extrêmement vigilant sur l'initialisation des variables de contrôle avant la boucle et leur modification explicite à l'intérieur du corps de la boucle pour éviter tout blocage indéfini.

- Pré-conditionnement : la condition booléenne est évaluée AVANT chaque exécution du bloc d'instructions.
- Plage d'itérations (0 à N) : si la condition est fausse dès l'entrée, le bloc d'instructions n'est jamais exécuté.
- Initialisation préalable : toutes les variables testées dans la condition du while doivent impérativement posséder une valeur définie avant d'entrer dans la boucle.

Cas concret enseignant : *Exemple du contrôle de code PIN au distributeur de billets : la machine demande le code TANT QUE le code saisi est incorrect ET QUE le nombre d'essais est inférieur à 3. Si l'usager tape le bon code dès le premier essai, la boucle s'arrête immédiatement sans répéter la demande.*

La boucle FAIRE... TANT QUE (do-while) : Structure à post-condition

À la différence de la boucle while, la structure do-while exécute toujours les instructions de son corps une première fois avant de contrôler la condition logique de sortie. Elle est le choix algorithmique naturel pour les contrôles de saisie (validation de bornes numériques) et la gestion des menus interactifs, où une action doit être proposée au moins une fois avant de vérifier si l'utilisateur souhaite poursuivre. L'oubli du point-virgule final de la ligne while (condition); est l'une des fautes de syntaxe les plus fréquemment commises par les étudiants et bloque systématiquement la compilation.

- Post-conditionnement : le test logique est réalisé APRÈS l'exécution du bloc d'instructions.
- Garantie d'exécution : la boucle s'exécute obligatoirement au minimum 1 fois (1 à N itérations).
- Syntaxe C stricte : obligation impérative de placer un point-virgule ';' immédiatement après la parenthèse fermante du while.

Cas concret enseignant : *Contrôle de saisie d'une note d'examen : 'Il est techniquement impossible de vérifier si la saisie faite par un étudiant est valide avant qu'il n'ait tapé au moins une valeur sur le clavier. On doit donc exécuter la saisie au moins une fois, ce qui fait du do-while la structure naturellement adaptée.'*

La boucle POUR (for) : Itération déterminée

La structure for rationalise la mécanique itérative en réunissant les trois composantes fondamentales d'une boucle sur une seule ligne d'en-tête : l'initialisation de la variable de contrôle, la condition logique de maintien et la mise à jour/incrémentation. Bien que le langage C permette d'omettre certaines expressions dans l'en-tête (comme dans for(;;) qui crée une boucle infinie), les renseigner explicitement est une exigence absolue de lisibilité et de propreté algorithmique.

- En-tête condensé : centralise l'initialisation, la condition de maintien et la mise à jour sous la forme for(init; cond; inc).
- Nombre d'itérations déterminé : réservée aux situations où le nombre exact de répétitions est connu avant d'entrer dans la boucle.
- Portée (scope) : possibilité de déclarer la variable de contrôle directement au sein de l'en-tête (ex: for(int i = 0; ...)).

Cas concret enseignant : *Métaphore des tours de piste d'athlétisme : 'Vous savez avant de vous lancer que vous devez effectuer exactement 5 tours. Vous initialisez votre compteur au tour 1 (init), vous vérifiez que vous n'avez pas dépassé 5 (cond), et à chaque franchissement de la ligne d'arrivée, vous ajoutez 1 à votre compteur (inc).'*

■ Questions & Interventions des Étudiants en Classe

Question étudiant : Pourquoi le programme plante ou bloque indéfiniment quand j'oublie d'incrémenter 'i' dans une boucle while ?
Réponse du professeur : Parce que la condition d'arrêt reste indéfiniment vraie (true). Le programme entre dans une boucle infinie qui consomme 100% du processeur et bloque l'exécution des instructions suivantes.

Question étudiant : Est-ce qu'on peut toujours remplacer une boucle 'for' par une boucle 'while' ?
Réponse du professeur : Oui, toute boucle for peut s'écrire sous forme de while en initialisant le compteur avant, en plaçant la condition dans le while et en faisant l'incrémentation à la fin du bloc d'instructions. C'est simplement une question de lisibilité et de structure.

■ Exercice & Démonstration d'Application

Énoncé : Écrire un programme complet en langage C qui calcule la somme des N premiers entiers positifs (ex: si N = 5, calculer 1 + 2 + 3 + 4 + 5 = 15). Le programme doit valider que N est un entier strictement positif (> 0) au

moyen d'une boucle 'do-while', puis calculer la somme cumulative à l'aide d'une boucle 'for'.

Démarche de résolution :
Étape 1 : Déclarer les variables entières n, somme (initialisée à 0) et i. Étape 2 : Mettre en place la saisie sécurisée du nombre N avec la structure do-while pour rejeter toute valeur négative ou nulle. Étape 3 : S'assurer que somme est initialisée à 0 avant le calcul. Étape 4 : Écrire la boucle for pour cumuler les valeurs de 1 jusqu'à N. Étape 5 : Afficher le résultat formaté avec printf.

```
/*
 * Fichier : somme_entiers.c
 * Auteur : Étudiant BUT Info Anglet
 * Description : Calcul de la somme des N premiers entiers avec validation do-while et boucle for
 */

#include <stdio.h>

int main()
{
    // CONSTANTES
    // (Aucune constante nécessaire pour cet exercice)

    // VARIABLES
    int n;
    int somme;
    int i;

    // TRAITEMENTS
    // Initialisation de la somme
    // (clavier) >> saisir >> n (avec validation do-while)
    somme = 0;
    do
    {
        printf("Entrez N (>0) : ");
        scanf("%d", &n);
    }
    while (n <= 0);

    // Calcul de la somme cumulative avec une boucle for
    for (i = 1; i <= n; i++)
    {
        somme += i;
    }

    // Affichage du résultat final
    printf("La somme des %d premiers entiers est %d\n", n, somme);

    return 0;
}
```

Résultat : Pour une saisie où l'utilisateur entre d'abord -3 puis 5 : la boucle do-while rejette -3 car (-3 <= 0) est VRAI, puis accepte 5. La boucle for réalise 5 itérations (somme prend successivement 1, 3, 6, 10, 15). Le programme affiche correctement : 'La somme des 5 premiers entiers est 15'.

■ Auto-Évaluation Flash d'Examen

Q : Quelle est la différence fondamentale d'évaluation entre une boucle 'while' et une boucle 'do-while' ?

R : La boucle while évalue sa condition en amont (pré-condition, 0 à N itérations), tandis que la do-while évalue sa condition en aval après un premier passage obligatoire (post-condition, 1 à N itérations).

Q : Que se passe-t-il si la condition initiale d'une boucle 'for' est fausse dès le premier test ?

R : Le corps de la boucle for ne s'exécute pas du tout (0 itération), exactement comme un while dont la condition de départ est fausse.

Q : Quel oubli de syntaxe systématique provoque une erreur de compilation sur la boucle 'do-while' en langage C ?

R : L'oubli du point-virgule ';' obligatoire juste après la parenthèse fermante de la condition while (ex: do { ... } while(cond);).